Managed database

The managed database is MySQL-compatible and requires `features.server: true`. Capability declaration defaults and enablement rules are in features. The platform supplies an empty database, not an ORM, application tables, migrations, or seed data.

Connection and data scope

`DATABASE_URL` is the connection DSN in URL form and is a server secret. Managed development and the published container connect to the same database: development writes and schema changes affect the data used by the published application.

The development sandbox supplies the DSN to the shell and dev process. Its `DRIZZLE_DATABASE_URL` alias has the same value; the published deployment supplies `DATABASE_URL`, not that alias. Local owns loading into a Local Host environment.

The database is project-scoped. The platform does not create an application-specific user, tenant schema, or row-ownership model.

Persistence, migration and recovery

A deployment or process restart does not create a fresh managed database. Existing schema, data, and migration history survive those events. Provisioning does not execute application migrations.

Separate process startup from one-time data preparation. Use the application's migration history
or seed version to run only unfinished work; record completion only after that work succeeds.
A non-empty table alone does not prove initialization completed. Preserve subsequent user edits rather than overwriting them with seed defaults on restart.

If startup depends on unfinished preparation, expose that state through application readiness
rather than guessing a fixed sleep. Container startup
owns the platform health budget. Migration, seed, and database-check scripts are finite tasks:
release their own connections and temporary resources after success or failure, so completed
work does not leave a process running solely because its clients remain open.

This platform surface provides no separate development database, production-to-development copy, point-in-time recovery, or full asynchronous backup/export API.

External database

With `features.database: false`, an application can supply an external `DATABASE_URL` through protected secret configuration. The config plane rejects simultaneous ownership of `DATABASE_URL` by the managed database and a user-defined secret. The platform does not choose the external database's schema or lifecycle.

Database implementation and change safety

Use the managed database by default for a server project's persistent application data. Choose `database: false` for a user-selected external database, or when the user explicitly chooses no database or a staged implementation without one; do not silently discard that choice. Explain the consequences when the requested behavior needs durable state that the selected design does not provide.

Read the DSN from the environment and use the TLS settings required by the selected driver. Keep migrations deterministic, repeatable, additive where possible, and committed with the application. Development and publication share the database, so treat development DDL and writes as changes to that same data.

Before destructive SQL, state what will be lost and obtain the user's approval for that destructive action. If a backup is required, use an available database-client or application export path; do not invent a platform restore endpoint. Check the affected application behavior after schema or migration changes.